Chess Game Analyzer, an introduction

David Joyner

2026-10-18

Contents

1 Introduction **9**

1.1 Computer chess for humans 9

1.2 The positional metrics 11

1.2.1 Data model and metric plots 11

1.2.2 Space 12

1.2.3 Mobility 12

1.2.4 King Safety 12

1.2.5 Threats 12

1.2.6 The Fireteam Index (experimental) 13

1.2.7 Balance 13

2 Examples **15**

2.1 Estrin-Berliner, 1965 15

2.1.1 Opening phase (Moves 1–5): The Two Knights Defense 16

2.1.2 The critical opening battle (Moves 6–15) 16

2.1.3 The blunder 16

2.1.4 The tactical middlegame (Moves 15–28) 17

2.1.5 The endgame, initial phase (Moves 28–34) 18

2.1.6 The final phase (Moves 35–42) 19

2.1.7 FTI summary 19

2.2 Carlsen-Caruana, Game 6, 2018 21

2.2.1 The early opening moves (1-4) 21

2.2.2 The opening phase (5-15) 24

2.2.3 The early middlegame (16-25) 24

2.2.4 The later middlegame (26-33) 25

2.2.5 The early endgame (34-67) 26

2.2.6 The later endgame (67-80) 28

2.2.7 FTI summary 30

3 Testing the Fireteam Index **31**

3.1 Does sustained dominance predict the result? 31

3.2 Do the FTI weights describe a playing style? 32

3.3 What helps and what does not 33

4 Appendix: Chess Game Analyzer module reference **35**
4.1 Overview . . . 35
4.2 Background for Newcomers . . . 35
4.2.1 What is Stockfish? . . . 35
4.2.2 What is a Centipawn? . . . 36
4.2.3 Python Requirements . . . 36
4.3 Installation . . . 36
4.4 Quick Start . . . 37
4.4.1 Command Line . . . 37
4.4.2 Python API . . . 37
4.5 Positional Metrics Explained . . . 38
4.5.1 Space . . . 38
4.5.2 Mobility . . . 38
4.5.3 King Safety . . . 39
4.5.4 Threats . . . 39
4.6 The Fireteam Index (FTI) . . . 40
4.6.1 Background: Chess Metrics and Win Prediction . . . 40
4.6.2 FTI Signatures . . . 40
4.6.3 Prediction Algorithms . . . 40
4.7 Concrete Example: Estrin vs Berliner (1965) . . . 41
4.7.1 The Game (C57 – Two Knights Defense) . . . 41
4.7.2 Analysis Output . . . 41
4.7.3 Positional Metrics Summary . . . 42
4.7.4 Move-by-Move Data Extract . . . 42
4.8 Command Line Interface . . . 42
4.8.1 Core Options . . . 43
4.8.2 Report Options . . . 43
4.8.3 Fireteam Index Prediction (Single Game) . . . 43
4.8.4 Book Mode (Multiple Games) . . . 43
4.8.5 Examples . . . 43
4.9 Python API . . . 44
4.9.1 Main Classes . . . 44
4.9.2 Utility Functions . . . 45
4.10 Parameter Reference . . . 46
4.10.1 Analysis Parameters . . . 46
4.10.2 FTI Prediction Parameters . . . 46
4.10.3 Available Weight Presets . . . 46
4.10.4 Move Classification Thresholds . . . 46
4.11 Output Formats . . . 47
4.11.1 LaTeX Report . . . 47
4.11.2 JSON Output . . . 47
4.11.3 Raw Positional Data . . . 48
4.12 License . . . 48
4.13 Acknowledgments . . . 48

5 References **49**

Preface

This is a guide to using a program I wrote to analyze chess games you input (as a pgn file on your computer) called the *chess game analyzer*, or *CGA* [CGA26]. The program uses other chess programs to output a math-dense LaTeX file, called a *Quantitative-Analytic Report on Chess* (or QARC), where the results from those other programs are collected in a human-readable format. Full disclosure: For creating CGA, I did get a lot of help from AI: `Gemini` (Google) for chess-theoretic questions, `ChatGPT` (OpenAI) for outline ideas, and `Claude` (Anthropic) for Python questions.

The CGA is a Python module providing an advanced chess analysis pipeline that goes beyond piece evaluations. It combines `Stockfish` engine analysis with "home-made" positional metrics to generate quantitative LaTeX reports on chess games, machine-readable data, and win-prediction metrics. In addition to generating educational content, it collects useful game data into tables, and computes an experimental "Fireteam Index" (or FTI, defined below). As we will see below, the FTI is a family of metrics inspired by the medieval board game rithmomachia [Ri25] to measure sustained positional dominance.

In fact, all this, the CGA module and related programs[1], was inspired by the "territorial" approach that proper victories take in rithmomachia. Can such an approach be helpful in chess analysis? That question is the main motivation for this work, and chapter 3 gives a partial answer: some rithmomachia principles help with chess and some do not.

Another goal was to create "Prediction Algorithms" for chess games in the hope of predicting a win during live play. Of course, that is impossible in general, but the question remains: what can be done, provided we allow some (not too many) inaccuracies? To address this, CGA implements three algorithms that use the FTI to predict game outcomes:

- `Per-Ply`.
  - Examines FTI at each ply (=half-move) after ply 16.
  - Predicts *win* if FTI $> \epsilon$ (default $\epsilon = 0.2$) for 10 consecutive plies.
  - Added plus: Captures sudden, decisive advantages.
- `Windowed`
  - This is a "smoothed" version of `Per-Ply`.
  - Computes a 10-ply rolling average of FTI, FTI_{10} say.

[1] For instance, CGA has an add-on that creates animated gifs of games, but such add-ons will be discussed separately.

  - Applies the same streak detection to smoothed values: Predicts *win* if $\text{FTI}_{10} > \epsilon$ for 10 consecutive plies.
  - Filters out tactical noise; captures sustained trends.

- `Balance`
  - Counts plies where FTI $> \epsilon$ versus FTI $< -\epsilon$
  - Measures volatility V (the number of times the FTI switches between favoring one side and favoring the other)
  - Computes confidence $= |B_n| \cdot \max(0,\, 1 - V/V_{\text{thr}})$, where V_{thr} is 0.2 times the number of plies considered
  - Predicts *draw* if confidence < 0.15; otherwise *win/loss* (that is, not a draw).
  - Best at detecting drawn games and balanced struggles

As chapter 3 explains, on the games tested so far these algorithms do not predict results better than simple baselines, such as always predicting the more common result, or the same rule applied to `Stockfish`'s evaluation.

The example games are well-known and well-analyzed, so you can compare CGA analysis here with the GM analysis published already in books, or blogs or on youtube. The first one is a World Correspondence Chess game played in 1965, Estrin-Berliner, §2.1. The second is Game 6 of the World Chess Championship match Carlsen-Caruana in London, Nov. 2018, §2.2.

You don't need to know how to use `Stockfish` or `Python` in order to run the CGA programs. However, you do need to know how to install them on your computer if they aren't already. Installation help and the syntax for using CGA is presented in detail in the appendix, ch. 4. Don't worry about the syntax or installing software yet. First, let's learn a little more about the CGA module.

Chapter 1

Introduction

As mentioned, detailed reference for the CGA module is given in an appendix, ch. 4. However, the basic idea is that the CGA module takes a pgn file as input. The output consists of a latex report which contains the following information about the game (or games) in the pgn file.

- Move-by-move positional analysis with Space, Mobility, King Safety, and Threats metrics
- `Stockfish` suggestions of playable alternative moves within 50 centipawns of the best
- Game character classification (balanced/tense/tactical/chaotic, one-sided/seesaw)
- Brilliant sacrifice detection with sound/speculative classification
- An experimental Fireteam Index (FTI) combining the four metrics, in five weightings
- Balance and Volatility, intended to detect drawn games
- Three experimental prediction algorithms (Per-Ply, Windowed, Balance) for win or draw predictions, with held-out tests in book mode showing how well they work
- Publication-quality LaTeX output with chess diagrams, plots, and methodology
- Book mode for analyzing multi-game PGN files with aggregate statistics
- Raw data export for downstream computational analysis

These will be explained, with examples, below.

1.1 Computer chess for humans

When a human plays chess in a given position, they use both logic (to trace through potential moves and the ramifications of their opponent's responses) and experience (to restrict their analysis to a small number of moves) in their effort to win. The human player makes a move based on rigorous reasoning and their previous experience with games similar to the

one they are currently playing. They pick one that, they feel, gives them the best position no matter what their opponent does in response. Of course, a player will not want to lose any advantage they feel they have in the position, whether material, or space, or mobility, or king safety, or potential threats they can mount. At the same time, they want their move to also restrict their opponent's advantages in the position.

A computer plays chess using a different process. The computer player also uses logic to sift through legal moves for the best one. The difference is that the computer can, with the help of a chess engine like `Stockfish`, evaluate positions very rapidly and they pick the move that both maximizes their own position evaluation and minimizes the evaluations of each response by their opponent. In other words, to a computer, playing chess is analogous to solving a math problem. Can humans learn a lesson or two from how the computer evaluates positions? Let's look more closely at this algorithmic perspective the computer player takes.

A real-valued function $m : \texttt{FEN} \to \mathbb{R}$ that takes in a position of a chess game and returns a real number is called a *chess metric*. Here, `FEN` simply denotes the collection of all possible FEN positions[1] and $\mathbb{R}$ is the standard notation for the set of real numbers.

One easy metric all players use is the *piece value metric* for White and for Black: $PV_W(\text{fen})$ denotes the total value of the white pieces in the position fen, and $PV_B(\text{fen})$ using the standard values for the Black pieces in the given position. Most players use the usual values[2] for the pieces:

piece	value
king	∞
queen	9
rook	5
bishop	3
knight	3
pawn	1

The reader can check that, in the game's starting position, not counting the kings, we have

$$PV_W(\text{start}) = PV_B(\text{start}) = 39.$$

Another example is a chess metric we'll denote by `eval`, the evaluation metric from the chess engine `Stockfish`. This positional metric measures advantages (including piece values) in *centipawns* (cp)—hundredths of a pawn. An evaluation of +100 (that is, $\texttt{eval}(\text{fen}) > 100$) means White is ahead in the given `FEN`position by the positional equivalent of roughly one pawn. An evaluation of -250 means Black has about a 2.5 pawn advantage.

[1] FEN stands for Forsyth–Edwards Notation, a standard notation for describing a board position, typically represented as an alphanumeric list or string, see [FEN]. Forsyth was a Scottish journalist from the 1800s while Edwards was an American computer scientist active in computer chess from the late 1900s to early 2000s.

[2] For more background on the different ways to evaluate chess pieces, and there are quite a lot of them, see the wikipedia article [PV] and the references there (including references to Berliner's book [Be99]).

Evaluation	Meaning
0 to ±50	Roughly equal
±50 to ±150	Slight advantage
±150 to ±300	Clear advantage
±300 to ±500	Winning advantage
±500+	Decisive advantage
±10000	Checkmate is forced

The modern `Stockfish` versions primarily use NNUE (neural network) evaluation to compute `eval`. Unfortunately, that metric is a bit of a black box. It appears to be impossible, or at least very impractical, to directly extract from the modern `Stockfish` other, more "territorial" metrics such as Space.

Let's dive into that topic next.

1.2 The positional metrics

The `Stockfish` evaluation `eval` tells the story, metrics explain why.

As mentioned, the modern `Stockfish eval` is a bit of a black box.

Given a chess position, how do we think of it from a territorial perspective? These natural questions arise:

- How do we compute the "space" controlled by a given side?
- How do we compute the "mobility" of pieces in the position?
- How do we compute the safety of the king? This is, once it's defined, a defensive metric.
- How do we compute the "threats" one side has against their opponent? Conversely, once it's defined, this is an offensive metric.

The four metrics Space/Mobility/King Safety/Threats use the Python module `python-chess` to parse the position and then Python to directly compute the metrics based on reasonable formulas[3]. These formulas are designed to address these questions.

1.2.1 Data model and metric plots

Raw positional data fields are printed at the bottom of the LaTeX QARC file, after `\end{document}`, so they do not appear in the compiled PDF. CGA uses this data to plot various chess metrics.

The FTI plots come in two versions, Per-ply and Windowed.

- Per-ply: Looks for 10 consecutive positions where one side dominates (FTI > 0.20).
- Windowed/Smoothed: Uses a 10-ply rolling average to filter out tactical noise.

[3]For details on `python-chess` and the formulas for the metrics (Space, Mobility, and so on), please see the following subsections or the appendix, chapter 4.

1.2.2 Space

"How much 'room' do your pawns give your pieces to move in the center?"

Space evaluation counts squares in the center (files c–f, ranks 2–4 for White, 5–7 for Black) that are:

1. Attacked by at least one friendly pawn, *and*
2. Not occupied by any enemy piece

The count is divided by 10, so Space ranges from 0 to 1.2 (there are 12 such squares per side); typical values are 0.1 to 0.7.

Space is a middlegame metric — it becomes near-meaningless in endgames when most pieces are traded.

1.2.3 Mobility

"How many useful squares can your pieces actually land on? (More squares = more power)"

Mobility adds 0.1 for every square attacked by one of your pieces (pawns included, the king excluded) that is *not* attacked by an enemy pawn. A square attacked by two of your pieces counts twice, and squares occupied by your own pieces count too, so Mobility rewards coordination as well as freedom of movement. All piece types are weighted equally. (QARC reports made before October 2026 show half of this value in their tables and plots, because of a since-fixed bug; see the appendix.)

Mobility tracks piece activity. Watch it climb during development and converge as long-range pieces (bishops, rooks, queens) are traded off.

1.2.4 King Safety

"How well is your King shielded by pawns, and how many nearby squares does the enemy attack?"

King safety combines two factors:

- **Pawn shield**: +0.5 for each friendly pawn on the 2nd rank (7th rank for Black) on the king's file or an adjacent file
- **Attacked squares near the king**: −0.2 for each square within two squares of the king (a 5×5 box) that an enemy piece attacks

King Safety can be positive and negative. Negative values in the middlegame mean danger. Negative values in the endgame can mean active, fighting kings.

1.2.5 Threats

"Are you hanging pieces, or are your small pieces attacking the opponent's big pieces?"

Threat evaluation considers:

- Hanging pieces (attacked but not defended)

- Pieces attacked by lower-value pieces (for example, a knight attacking a rook)
- Attacks on the squares around the enemy king
- Safe checks available
- Weak squares (holes) in the enemy pawn structure

Threats measures potential piece captures. High threat values show tension; when they drop sharply after exchanges, material or positional concessions have been made.

Example 1.2.1. *For example, the $-328cp$ blunder at move 15 in §2.1.2 below shows what happened but the Mobility and Threats changes in moves 8-14 show why the position was ripe for sharp play from both sides.*

1.2.6 The Fireteam Index (experimental)

This is a weighted sum of the differences (player minus opponent) of the four metrics above.

The Fireteam Index is a composite metric inspired by the ancient mathematical board game rithmomachia, where a "progression victory" is achieved when a coordinated group of pieces (a "fireteam") establishes control deep in enemy territory [Ri25]. Similarly, in chess, sustained positional dominance across multiple factors often precedes victory.

CGA provides five weightings of the FTI, named for the kind of play they emphasize (see the appendix). The original hope was that these weightings would act as *signatures* of different playing styles, and that a player who keeps the FTI positive for long enough is on the way to winning. Chapter 3 tests both ideas; on the games tested so far, neither holds up.

1.2.7 Balance

Balance is the key draw-detection metric. It combines:

- Normalized Balance (B_n): the number of plies with FTI $> \epsilon$, minus the number with FTI $< -\epsilon$, divided by the number of plies considered
- Volatility (V): How often the advantage changed hands
- Confidence (C): $C = |B_n| \cdot \max(0,\, 1 - V/V_{\text{thr}})$, where V_{thr} is 0.2 times the number of plies considered.

The idea is that if the positional metrics are roughly equal for most of the game then neither side dominated the Space/Mobility/Threats picture consistently. Similarly, high volatility means the advantage kept changing hands during the course of the game. The *Balance algorithm* will predict the game ends in a draw if $C < 0.15$; otherwise it predicts a win for the side favored by B_n. This data is part of the QARC output.

Chapter 2

Examples

The first example game, ironically, was won by Hans Berliner, a highly respected computer pioneer and computer science researcher.

2.1 Estrin-Berliner, 1965

This is a popular game for commentary and analysis. The point here is not to try to provide a summary of the analysis of others but to report on what CGA says about these games.

The QARC file on the game Estrin vs. Berliner (1965) describes it as a "chaotic seesaw battle," where the advantage changed hands multiple times [Be65]. We'll walk you through this game using the provided metrics, you can use them to explain the dynamics of the positional components (Space, Mobility, King Safety, and Threats).

White, played by the Russian chess player Yakov Estrin, took joint first place in the USSR Correspondence Championship a few years earlier, in 1962. Estrin is best known for being the seventh ICCF World Champion a few years later, in 1972–1976.

Black, played by American Hans Berliner, was also a well-known chess player. Here is a quotation from Dunne's remembrance article for Hans Berliner written for **Chess Life** [Du17]:

> The finals of the Fifth World Correspondence Chess Championship produced a score of $14 - 2$, 12 wins and four draws, a three point margin over the second place finisher, a spread unmatched in World Championship play. In that final, Hans also played what is widely regarded as the greatest correspondence game ever played, and listed in Andy Soltis' book **The 100 Best Chess Games of the 20th Century** as the top game of that tumultuous century. It is a game that has resisted major improvement on Black's part for over half a century.

In fact, that last statement of Dunne's is, thanks to progress in chess engines such as `Stockfish`, no longer completely true. However, we aren't going down that rabbit hole in this guide.

2.1.1 Opening phase (Moves 1–5): The Two Knights Defense

With that background out of the way, let's see what the CGA had to say about this game.

1.e4 e5 2.Nf3 Nc6 3.Bc4 Nf6 4.Ng5 d5 5.exd5

This is the famous Two Knights Defense with the Fried Liver Attack. Estrin plays 4.Ng5, immediately creating tension. Let's see how the metrics capture this.

Notice how Space stays roughly equal, between 0.30 and 0.40 for both sides, through move 9. Space measures control of central squares backed by your pawns—essentially, how much room your pieces have to maneuver. After

5.exd5

White captures a pawn but Black will strike back. Watch what happens after 5...b5—the Ulvestad Variation. The usual move in this position is 5. ... Na5.

Data at ply 10/move 5: Eval: +0.55

Look at the Threats metric—White's threats spike to 10.80 because the b5-pawn is hanging. But if you compare Black's Mobility to White's, you see Berliner is accepting material deficit for piece activity.

2.1.2 The critical opening battle (Moves 6–15)

6. Bf1 Nd4 7. c3! Nxd5! 8. Ne4 Qh4?

From the game's annotation by `Stockfish`, both 8 ... Nc6 and 8 ... Ne6 are preferred.

9. Ng3! Bg4

This time `Stockfish` has no objection to Black's move.

Data at ply 18/move 9:
Eval: +0.99
The game goes on (see [Be65]):

10. f3 e4??

Black loses 139cp. `Stockfish` recommends 10...Bd7 instead.

11. cxd4 Bd6 12. Bxb5+? Kd8?! 13. O-O exf3! 14. Rxf3! Rb8!

`Stockfish` prefers 12. Qe2 to 12. Bxb5+. The position after 14...Rb8 is a crucial one.

2.1.3 The blunder

With threats from both sides, the next move is important.

15. Be2??

Looks good but, in fact, some commentators say this natural-looking move is the move that lost the game. `Stockfish` recommends 15. Bf1 instead; 15. Be2 loses about 330 centipawns.

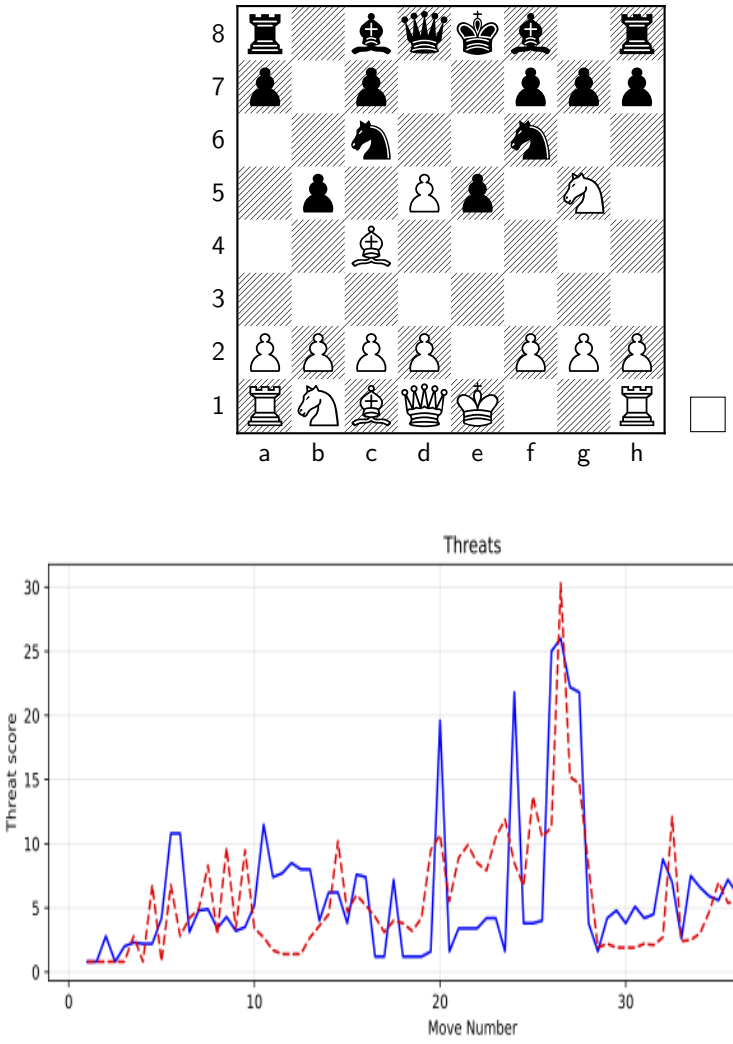


Figure 2.1: (top) Position after 5 ... b5 in Estrin-Berliner (1965). (bottom) The Threats metric for that game.

Data at ply 29/move 15:
Eval: -1.57

2.1.4 The tactical middlegame (Moves 15–28)

The game continues:

16. Bxf3! Qxd4+?

`Stockfish` preferred instead 16... Re8.

17. Kh1! Bxg3! 18. hxg3! Rb6 19. d3! Ne3 20. Bxe3 Qxe3! 21. Bg4 h5! 22. Bh3! g5 23. Nd2 g4 24. Nc4! Qxg3 25. Nxb6 gxh3 26. Qf3 hxg2+ 27. Qxg2!

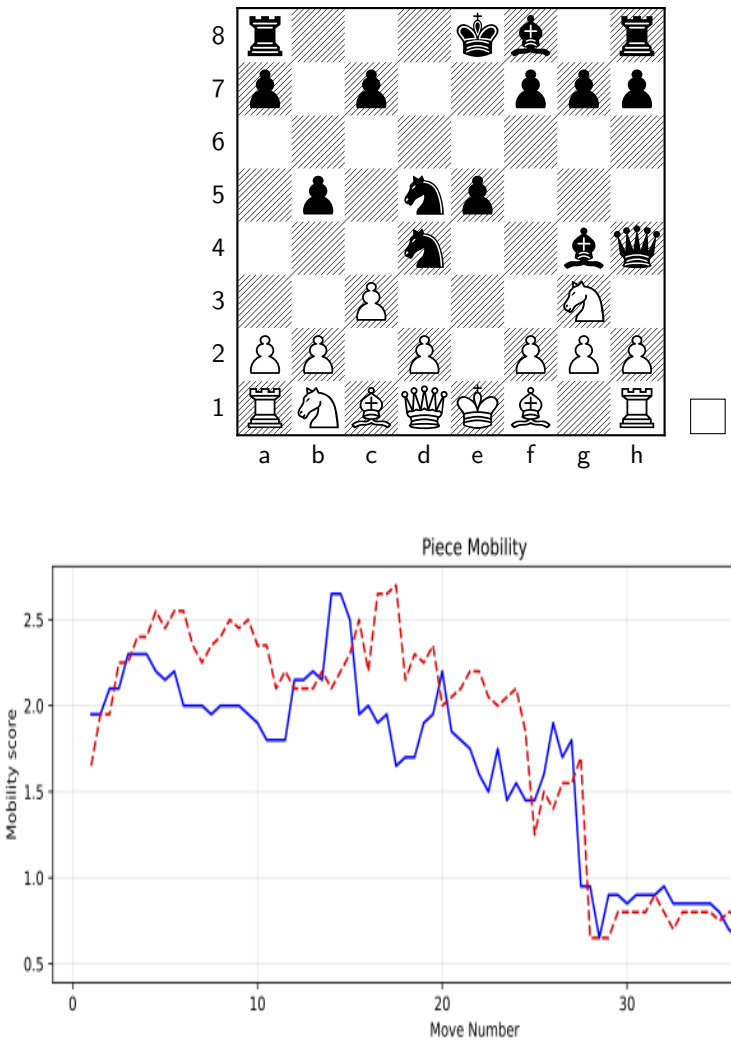


Figure 2.2: (top) Position after 9 ... Bg4 in Estrin-Berliner (1965). (bottom) The Mobility metric for that game.

Qxg2+!

The queens are trading off now.

2.1.5 The endgame, initial phase (Moves 28–34)

Without long-range pieces, the Mobility metric drops. We enter a new phase of the endgame.

28. Kxg2! cxb6 29. Rf1 Ke7 30. Re1+ Kd6 31. Rf1 Rc8?

`Stockfish` recommends 31...Ke6 or 31...Ke7 instead.

Data at ply 62/move 31:

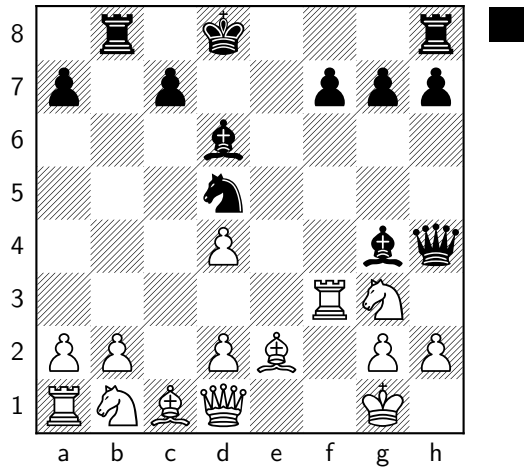


Figure 2.3: Position after 15 Be2 in Estrin-Berliner (1965).

Eval: -1.28

32. Rxf7 Rc7 33. Rf2?

`Stockfish` suggests 33. Rf5 instead.

33...Ke5 34. a4??

This is a more serious error, but not of the same magnitude as 15 Be2??. With 34. a4??, `eval` drops from -1.90 to -3.52. Here, `Stockfish` recommends instead Rf8.

34...Kd4! 35. a5 Kxd3

2.1.6 The final phase (Moves 35–42)

The final moves of the game.

36. Rf3+! Kc2! 37. b4?! b5! 38. a6? Rc4?! 39. Rf7 Rxb4! 40. Rb7? Rg4+! 41. Kf3

`Stockfish` flags 38. a6? (38. Rf5 was better) and 40. Rb7? (40. Rxa7 was better).

41...b4! 42. Rxa7 b3! 0-1

2.1.7 FTI summary

Recall that the FTI is a weighted sum of the differences (player minus opponent) in the four positional metrics, Space (S), Mobility (M), King Safety (KS), and Threats (T):

$$FTI = w_S\,\Delta S + w_M\,\Delta M + w_{KS}\,\Delta KS + w_T\,\Delta T',$$

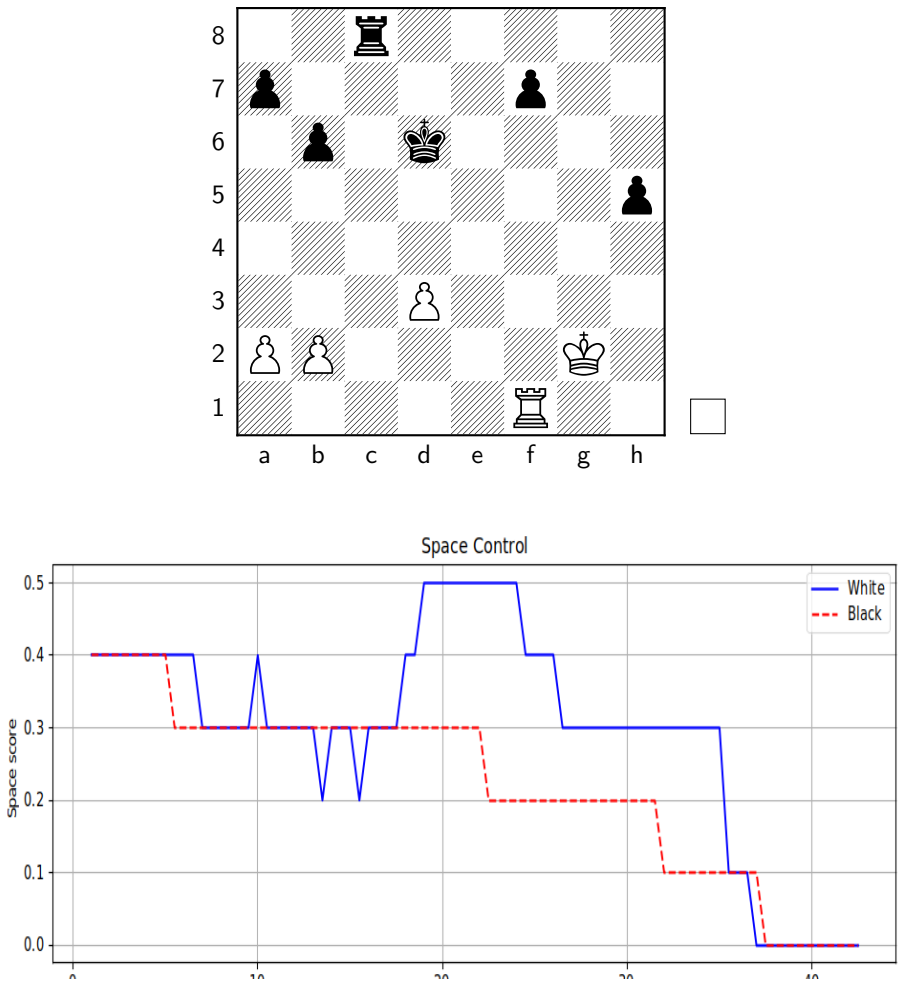


Figure 2.4: (top) Position after 31 ... Rc8? in Estrin-Berliner (1965). (bottom) The Space metric for that game.

where $T' = T/20$ (see the appendix). Think of the FTI as a coordinated pressure gauge. Figure 2.5 shows FTI1, which weights the four metrics equally.

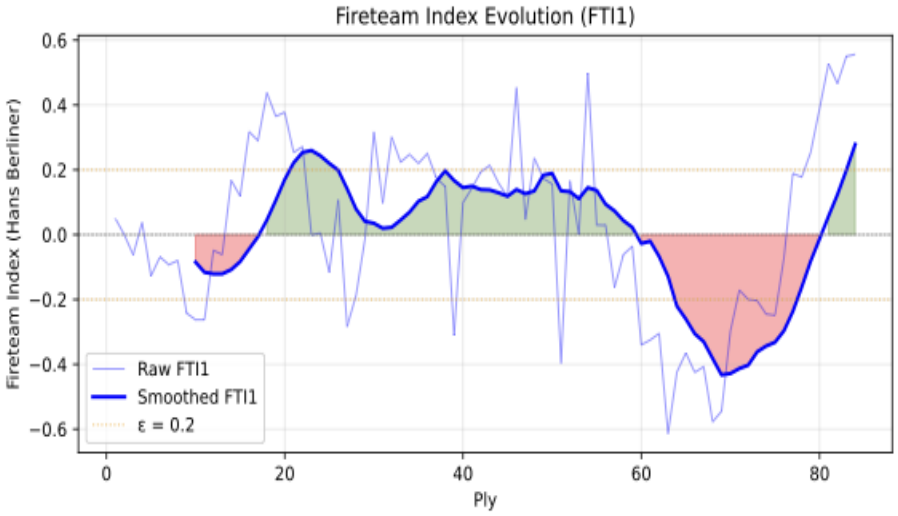


Figure 2.5: Fireteam index FTI1 (equal weights) for Estrin-Berliner, 1965.

The game was so volatile that the FTI swings back and forth. With the current version of CGA, the Per-ply rule (dominance for 10 consecutive plies) does not predict Black's win with any of the five weightings; the Windowed rule does with FTI1 and FTI4. More importantly, across all 16 of Berliner's games in this championship, the tests in chapter 3 find no evidence that the FTI predicts results better than simple baselines, or that any weighting of it describes Berliner's style.

2.2 Carlsen-Caruana, Game 6, 2018

At this point in the match, the players were tied 2.5-2.5, with 5 draws in a row. This game was another draw but exciting nonetheless. In fact, on move 67 it appears Caruana missed a win.

The QARC file on the 12 games in the Carlsen-Caruana match, London (November 2018) is available online as a pdf [CC18]. We'll walk you through this game using the provided metrics, you can use them to explain the dynamics of the positional components (Space, Mobility, King Safety, and Threats).

Of course, both players are well-known. In 2018, the 27-year-old Norwegian was the undisputed king of chess, having held the World Championship title since 2013 and the world No. 1 ranking since 2011. The 26-year-old Italian-American grandmaster, in 2018, became the first American to challenge for the undisputed world chess title since Bobby Fischer in 1972. At the time of their match, their Elo ratings were nearly the same.

2.2.1 The early opening moves (1-4)

Look at the plot for `Stockfish`'s `eval` function (see Figure 2.6).

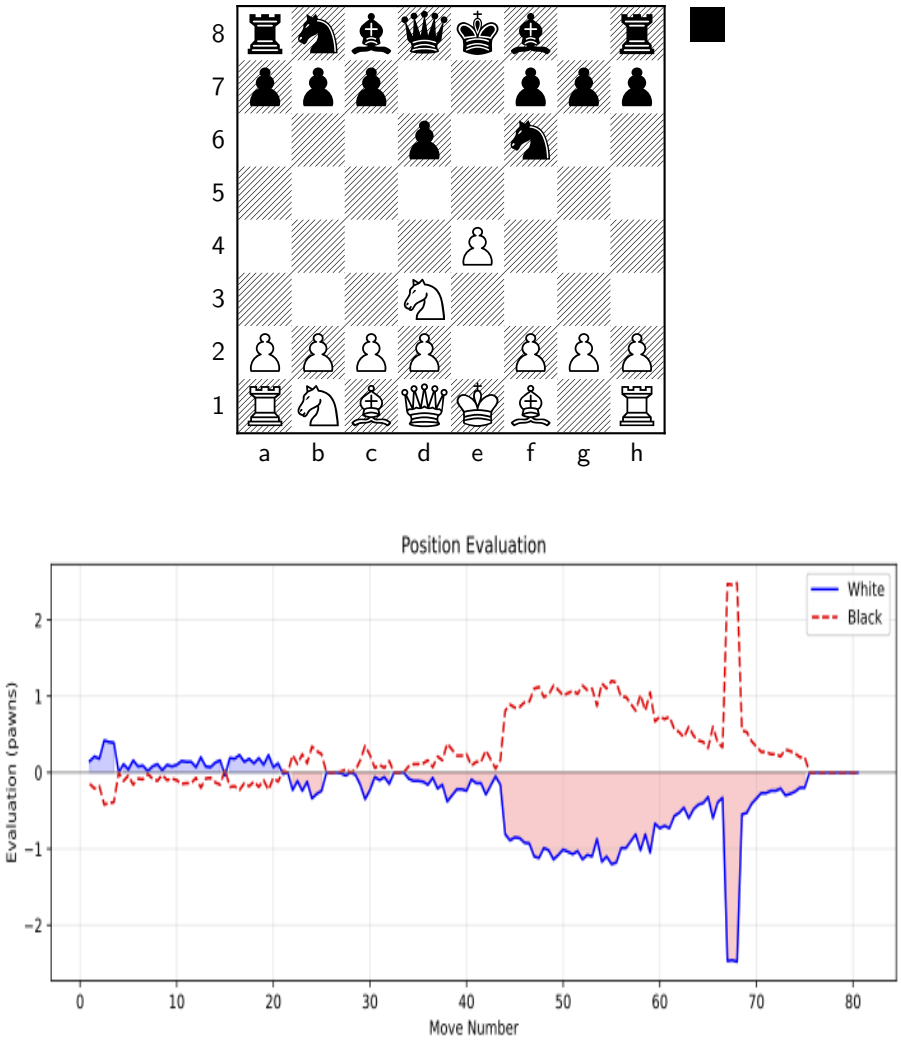


Figure 2.6: (top) Position after 4 Nd3?! in Carlsen-Caruana, Game 6 (2018). (bottom) The `eval` metric for that game.

Notice how the line hovers around zero for the first 10-15 moves, then drifts into negative territory, favoring Caruana. The game reaches its most critical point around moves 67-68 where we see a dramatic spike to about -2.3 pawns, a large advantage for Black. Unfortunately for Caruana, that was followed by an equally dramatic swing back toward zero.

1. e4 e5 2. Nf3 Nf6 3. Nxe5 d6 4. Nd3?!

While this is part of opening theory[1], `Stockfish` marks Carlsen's move as an inaccuracy (?!).

[1]ECO C42: Petrov's Defence, Karklins-Martinovsky Variation, to be precise.

Position 1: After 4. Nd3
Evaluation: +0.08

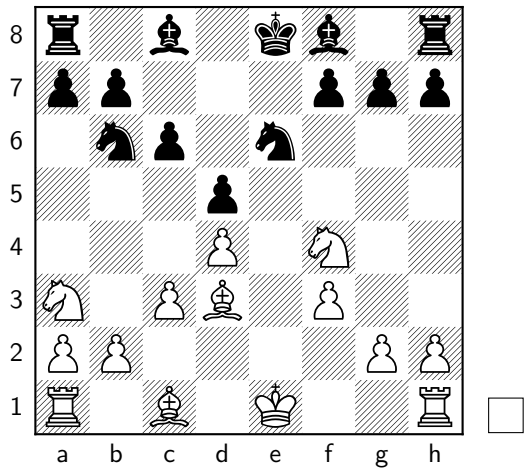
2.2.2 The opening phase (5-15)

4 ... Nxe4 5. Qe2 Qe7 6. Nf4 Nc6 7. Nd5! Nd4! 8. Nxe7 Nxe2! 9. Nd5! Nd4! 10. Na3 Ne6! 11. f3! N4c5 12. d4! Nd7 13. c3 c6! 14. Nf4! Nb6! 15. Bd3 d5

This is all part of the opening. White is given a slight advantage by `Stockfish`.

Figure 2.7 tells us Caruana successfully seized territorial control (Space) in the opening up to move 15. In Figure 2.8, look at how the Mobility values drop after the unusual early queen trade on move 8.

Position 2: After 15. ... d5



After **15. ... d5**.

2.2.3 The early middlegame (16-25)

The game continued:

16. Nc2! Bd6 17. Nxe6 Bxe6 18. Kf2! h5! 19. h4! Nc8 20. Ne3 Ne7 21. g3! c5! 22. Bc2 O-O 23. Rd1 Rfd8 24. Ng2 cxd4! 25. cxd4 Rac8

This ends the early middlegame. The game is even, according to `Stockfish`.

Position 3: After 25. ... Rac8

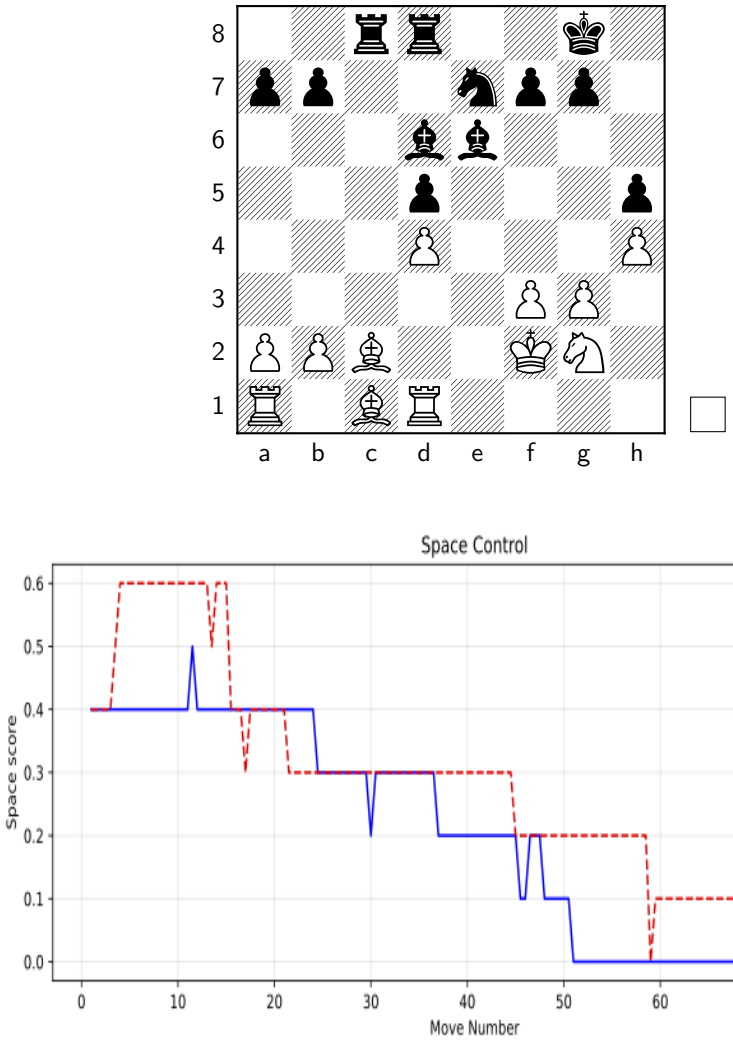


Figure 2.7: (top) Position after 25 ... Rac8 in Carlsen-Caruana, Game 6 (2018). (bottom) The Space metric for that game.

2.2.4 The later middlegame (26-33)

The game continued:

26. Bb3! Nc6 27. Bf4 Na5! 28. Rdc1! Bb4 29. Bd1?! Nc4 30. b3! Na3 31. Rxc8! Rxc8! 32. Rc1! Nb5 33. Rxc8+! Bxc8!

Position 4: After 33. ... Bxc8!

Note how the Mobility metric drops after these piece exchanges.

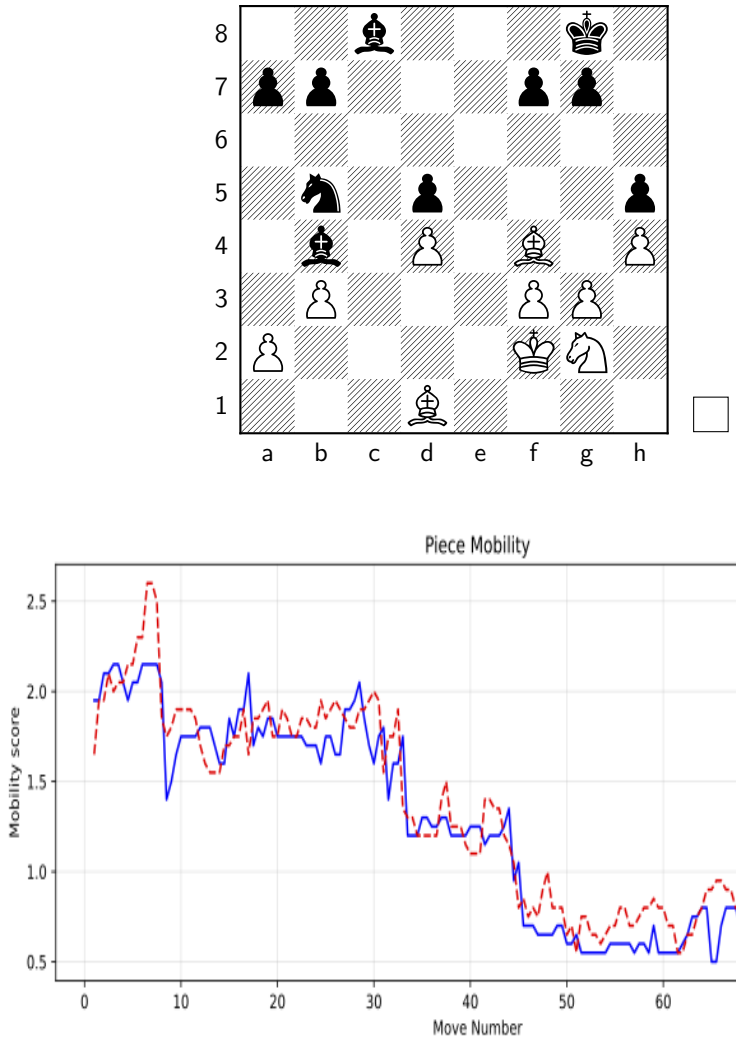


Figure 2.8: (top) Position after 33 ... Bxc8 in Carlsen-Caruana, Game 6 (2018). (bottom) The Mobility metric for that game.

2.2.5 The early endgame (34-67)

The game continued:

34. Ne3 Nc3! 35. Bc2! Ba3! 36. Bb8 a6 37. f4 Bd7 38. f5?! Bc6! 39. Bd1! Bb2 40. Bxh5 Ne4+ 41. Kg2! Bxd4! 42. Bf4 Bc5! 43. Bf3! Nd2

This is a very interesting move, setting in motion a long combination, dropping piece Mobility but escalating Threats.

Position 5: After 43. Bf3

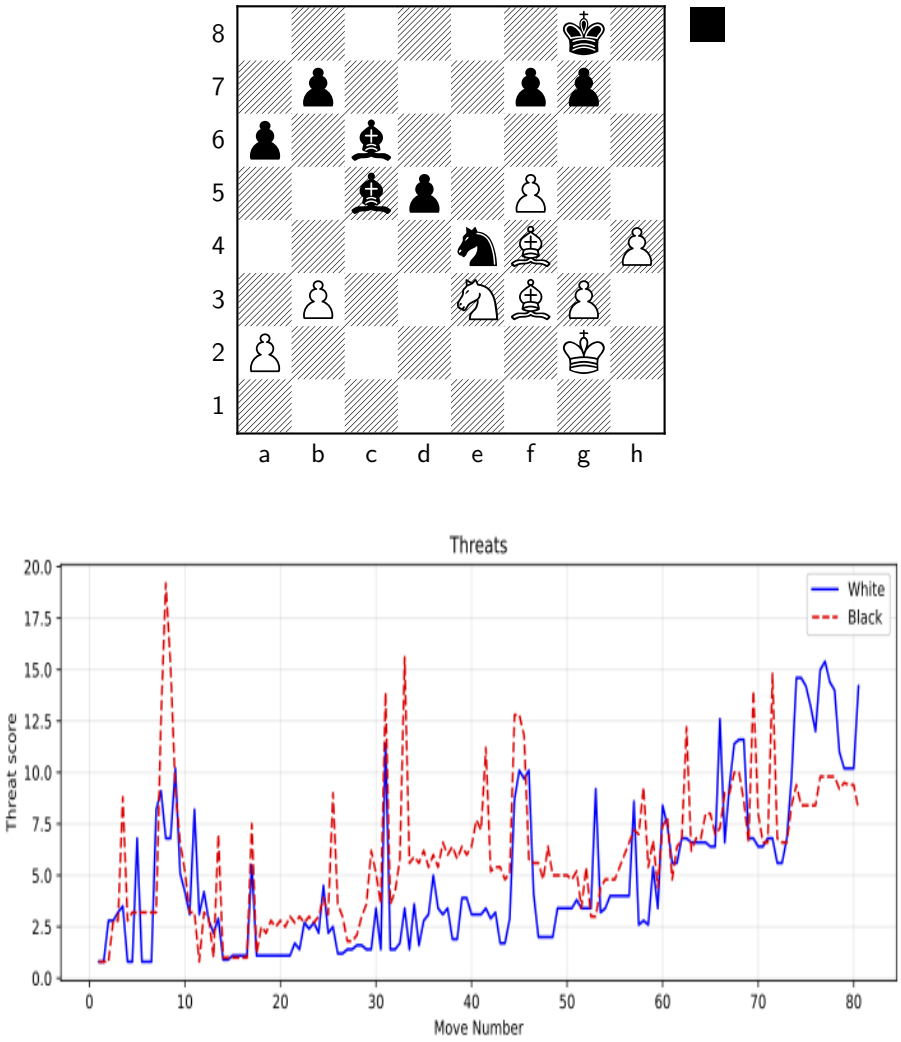


Figure 2.9: (top) Position after 43 Bf3 and before 43 ... Nd2! in Carlsen-Caruana, Game 6 (2018). (bottom) The Threats metric for that game.

44. Bxd5?

`Stockfish` didn't like Carlsen's move and suggests considering Nf1 or Nd1 instead.

Position 6: After 44. Bxd5
Evaluation: -0.80

The game continued:

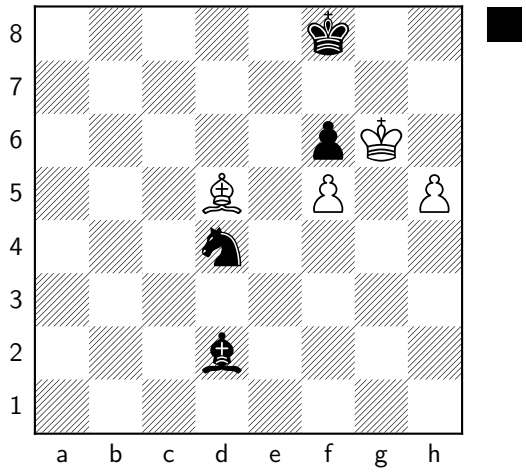
44...Bxe3! 45. Bxc6! Bxf4! 46. Bxb7! Bd6 47. Bxa6 Ne4! 48. g4! Ba3! 49. Bc4 Kf8 50. g5 Nc3! 51. b4 Bxb4 52. Kf3 Na4! 53. Bb5 Nc5?! 54. a4?! f6 55.

Kg4! Ne4 56. Kh5 Be1 57. Bd3?! Nd6 58. a5 Bxa5! 59. gxf6! gxf6! 60. Kg6! Bd8 61. Kh7 Nf7?! 62. Bc4! Ne5 63. Bd5! Ba5 64. h5! Bd2 65. Ba2 Nf3 66. Bd5 Nd4 67. Kg6?!

`Stockfish` marks this as an inaccuracy: after 67...Bg5, Black is more than two pawns ahead.

Position 7: After 67. Kg6

Evaluation: -1.35



2.2.6 The later endgame (67-80)

After 67...Bg5, `Stockfish` rates Black's advantage at -2.27, the largest of the game.

67...Bg5! 68. Bc4 Nf3??

According to `Stockfish`, this is a blunder; it recommends Bh4 instead. According to `Stockfish`, with depth=23, with this knight move Caruana missed a win.

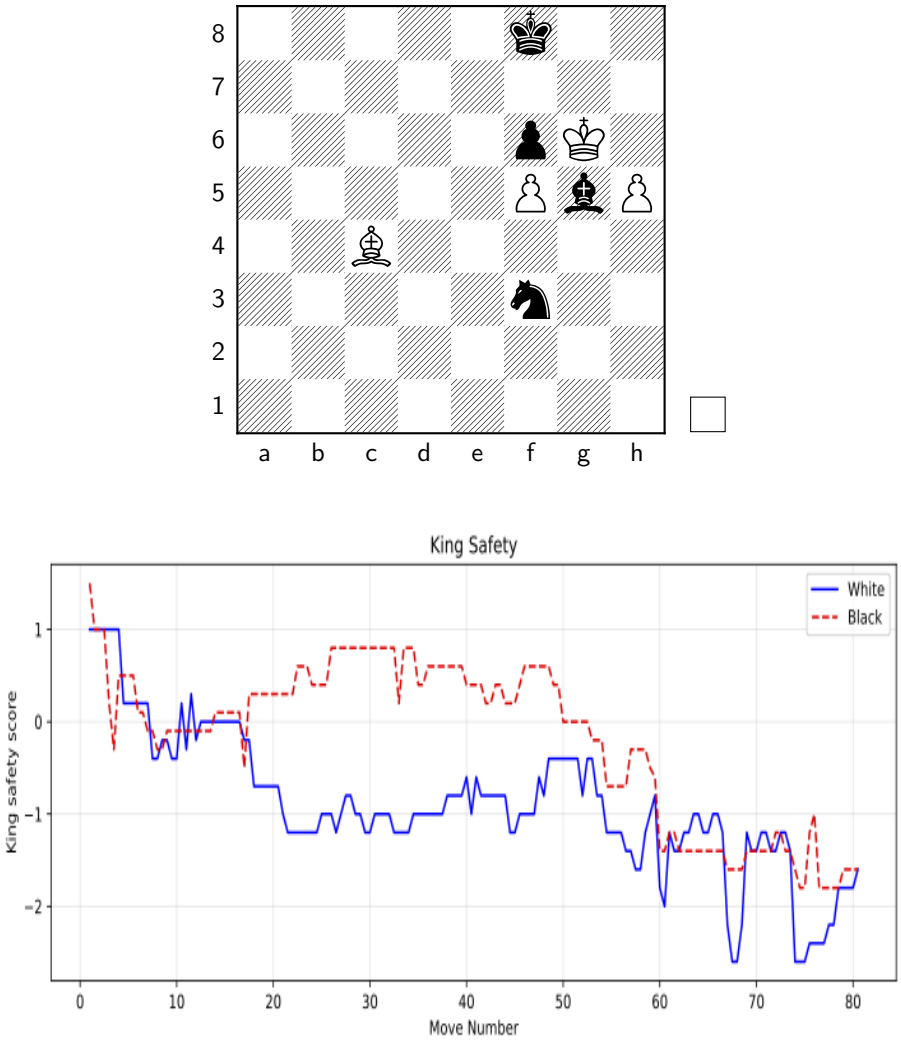


Figure 2.10: (top) Position after 68 ... Nf3?? (moving from d4) in Carlsen-Caruana, Game 6 (2018). (bottom) The King Safety metric for that game.

After 18 Kf2, the White king enters the game early instead of castling, which here created the opportunity for Black to make more threats. Based on Figure 2.10 (bottom), note the King Safety line drops into negative territory (-0.7 to -1.2) after the opening and stays there. On the other hand, Black's King Safety was positive from moves 18 to 50 (after 49 ... Kf8). Since Carlsen's king was more exposed to checks and needed protection, Caruana was able to keep stronger positional evaluations, even with equal material.

The game concluded this way:

69. Kh7 Ne5 70. Bb3 Ng4 71. Bc4 Ne3! 72. Bd3! Ng4 73. Bc4 Nh6 74. Kg6 Ke7 75. Bb3! Kd6?! 76. Bc2! Ke5 77. Bd3 Kf4 78. Bc2 Ng4 79. Bb3 Ne3?! 80. h6! Bxh6! 1/2-1/2

2.2.7 FTI summary

Figure 2.11 shows FTI1 (equal weights) for this game.

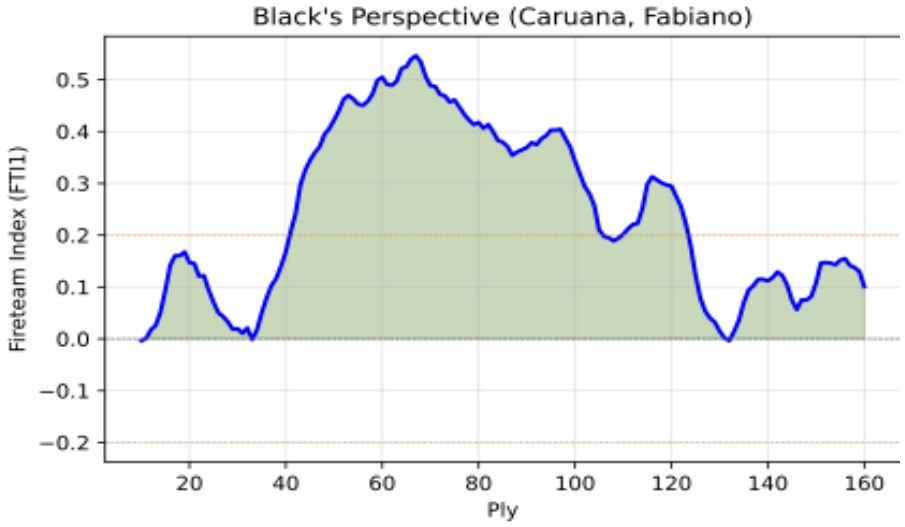


Figure 2.11: Fireteam index FTI1 (equal weights) for game 6 of Carlsen-Caruana, 2018.

All twelve classical games of this match were drawn, so the FTI cannot be tested as a predictor of results on this match: there is no variation in results to learn from. (An earlier version of this manual reported "optimal" weights $(0, 0, 0, 1)$ with $R^2 = 0$ for this match; those numbers were an artifact of fitting weights to twelve identical results.)

Chapter 3 asks a different question of this match: given several near-equal moves, do Carlsen and Caruana choose differently, as measured by the four metrics? They do not. A model fitted to Caruana's choices predicts Carlsen's as well as Carlsen's own model does, and vice versa.

Chapter 3

Testing the Fireteam Index

The FTI was built on a rithmomachia idea: a coordinated group of pieces (a "fireteam") that establishes control deep in enemy territory, and holds it, is on the way to victory. This chapter tests two consequences of that idea on the games in this manual:

1. sustained FTI dominance predicts the result of a game, and

2. the FTI weights capture a player's style, so they could be used to predict the kind of move a player will choose.

The short answer is that some rithmomachia principles help with chess and some do not.

3.1 Does sustained dominance predict the result?

Berliner scored 12 wins and 4 draws in his 16 games of the 5th World Correspondence Championship [Be65]. Table 3.1 compares the FTI prediction rules with two baselines: always predicting a Berliner win, and the same 10-ply streak rule applied to `Stockfish`'s evaluation (Berliner more than one pawn ahead). The last row refits the FTI weights 16 times, each time leaving one game out and predicting it. The "first half" column uses only the first half of each game, which is the useful case for prediction: by the end of a game, the evaluation nearly shows the result. All numbers were computed with the current version of CGA from the raw data in [Be65].

Method	Whole games	First half only
Always predict a Berliner win	12/16	12/16
Streak rule on `Stockfish`'s evaluation	15/16	8/16
FTI1 (Harmonious), Per-ply rule	7/16	6/16
FTI2 (Tactical), Per-ply rule	7/16	6/16
FTI3 (Strategic), Per-ply rule	8/16	6/16
FTI4 (Dynamic), Per-ply rule	11/16	8/16
FTI5 (Solid), Per-ply rule	7/16	5/16
FTI weights refit with the game held out	13/16	11/15*

Table 3.1: Correct predictions of Berliner's results. *One game's first half is entirely within the first 16 plies, which the FTI ignores.

No FTI rule beats simply predicting a Berliner win every time. The FTI is also only weakly related to the evaluation: across all positions after ply 16, the correlation between FTI1 and `Stockfish`'s evaluation is 0.28. For the 2018 Carlsen-Caruana match no such test is possible, because all twelve games were drawn.

In book mode, CGA now runs the held-out test (the last row) on every collection, in place of the "empirically optimal weights" that earlier versions printed. Those weights and their R^2 values (for example, $R^2 = -12.5$ for Berliner's games) were not meaningful: the fit had no constant term, and its weights were clipped and rescaled afterwards.

3.2 Do the FTI weights describe a playing style?

Results are a poor guide to style: they mostly reflect who made fewer mistakes. A better guide is the moves a player chooses. In many positions, `Stockfish` rates several moves as nearly equal; which one a player picks is a matter of taste. If Berliner keeps choosing the move that raises his Mobility over the one that gains Space, that *is* his style.

Method. For every position after move 8 in Berliner's 16 games and the 12 games of the 2018 match[1], `Stockfish` 16 (depth 14) listed its six best moves. The *candidates* are the moves within 50 centipawns of the best. That leaves 1,455 positions with a real choice, after dropping 370 positions with only one playable move and 91 where the move played was more than 50 centipawns worse. For each candidate, CGA computed the change in Space, Mobility, King Safety and Threats (player minus opponent). A standard statistical model of choices (a conditional logit) then estimates how much each metric, and the evaluation, sways the player's choice. Every model was tested only on games it was not fitted on.

[1]With the colors of the actual games. Before October 2026, the QARC file [CC18] listed Carlsen as White in all twelve games, which was wrong for games 1, 3, 5, 8, 10, and 12; the current version is correct.

Player	Choices	Random pick	Eval only	Eval + 4 metrics
Berliner	281	31%	60%	57%
Berliner's opponents	276	29%	50%	48%
Carlsen	459	22%	49%	46%
Caruana	439	24%	53%	49%

Table 3.2: How often each model's most likely move was the move actually played, on held-out games.

The results (Table 3.2) are clear:

- Adding the four metrics to the evaluation makes the predictions slightly *worse* for every player, so the metrics do not reveal a personal style.
- Carlsen and Caruana cannot be told apart: a model fitted to Caruana's choices predicts Carlsen's as well as Carlsen's own model does, and vice versa.
- Berliner's own model does beat his opponents' model, but only because he picks `Stockfish`'s preferred move more often. That is playing strength, not style.

3.3 What helps and what does not

There is one consistent pattern, and every group in the study shares it. Among near-equal moves, players pick the candidate that most increases their Threats, Mobility, or King Safety a few percentage points more often than a random choice would (Table 3.3). Space barely matters.

Candidate scoring highest on	Excess over random choice	90% interval
`Stockfish`'s evaluation	+25.1 points	+23.0 to +27.4
Threats	+6.3 points	+4.6 to +8.0
Mobility	+4.7 points	+3.1 to +6.5
King Safety	+4.4 points	+2.4 to +6.4
Space	+1.7 points	+0.8 to +2.7

Table 3.3: How much more often than random the move played was the candidate scoring highest on each measure, over all 1,455 choices (intervals by resampling games).

Putting the two studies together:

- **Principles that help.** Coordination and activity (Mobility) and pressure on the enemy (Threats), the "fireteam" half of the rithmomachia idea, describe how strong players choose among equally good moves. They are shared by all the players studied, so they describe good chess rather than one player's style, and they are too weak to

improve predictions once `Stockfish`'s evaluation is known. The metric plots in the examples above also help explain *why* a game went the way it did.

- **Principles that do not help.** Territory as such (Space) barely influences the choices of strong players. Holding positional dominance as a winning condition (the FTI prediction rules) does not predict results better than simple baselines. And no weighting of the FTI acts as a signature of a player's style.

These are small samples (fewer than 500 choices per player), the four metrics are simple, and the candidate moves came from a modest search depth. Better metrics might find something these four do not, and the tests above, which CGA now runs on every book, are the way to find out.

Chapter 4

Appendix: Chess Game Analyzer module reference

This is the readme file for the CGA program [CGA26] generated by `Claude` (Anthropic), with some editing by me.

4.1 Overview

The Chess Game Analyzer is a Python module for deep positional analysis of chess games using Stockfish, with comprehensive LaTeX report generation and win prediction metrics.

Chess Game Analyzer goes beyond simple engine evaluation to provide interpretable positional metrics for every move of a chess game. It computes Space control, Mobility, King Safety, and Threats directly from board positions, then combines these into **Fireteam Index (FTI)** metrics that can predict game outcomes based on sustained positional advantages.

The module generates professional LaTeX reports with annotated games, critical position diagrams, matplotlib plots, and detailed methodology explanations—suitable for chess study, coaching, or research.

4.2 Background for Newcomers

4.2.1 What is Stockfish?

`Stockfish`[1] is the world's strongest open-source chess engine. It analyzes positions and calculates the best moves by searching millions of positions per second. This module uses Stockfish to evaluate positions, but computes the positional breakdown (Space, Mobility, etc.) directly using the `python-chess` library.

Stockfish must be installed on your system. Download it from the official website and note the installation path.

[1] `https://stockfishchess.org/`

4.2.2 What is a Centipawn?

Chess engines measure advantages in **centipawns (cp)**—hundredths of a pawn. An evaluation of +100 means White is ahead by roughly one pawn's worth of material or positional compensation. An evaluation of −250 means Black has about a 2.5 pawn advantage.

Evaluation	Meaning
0 to ±50	Roughly equal
±50 to ±150	Slight advantage
±150 to ±300	Clear advantage
±300 to ±500	Winning advantage
±500+	Decisive advantage
±10000	Checkmate is forced

4.2.3 Python Requirements

The module requires:

- **Python 3.8 or higher**
- **python-chess** (installed as `chess`): The chess library that handles board representation, move generation, and PGN parsing
- **matplotlib** (optional): For generating plots; the module works without it using ASCII plots. Plots also need the file `chess_plotting.py` in the same folder as `chess_game_analyzer.py`.

To install the dependencies:

```
pip install chess matplotlib
```

4.3 Installation

1. **Install Stockfish** from `https://stockfishchess.org/download/`
2. **Clone the repository** `https://github.com/wdjoyner/chess-animator` or download `chess_game_analyzer.py` and `chess_plotting.py` from it. Keep both files in the same folder: without `chess_plotting.py`, reports are produced with no plots.
3. **Install Python dependencies**:

   ```
   pip install chess matplotlib
   ```

4. **Verify Stockfish path**:

   ```
   # Linux/Mac - find where Stockfish is installed
   which stockfish

   # Or test directly
   stockfish
   ```

4.4 Quick Start

4.4.1 Command Line

```
# Analyze a single game
python chess_game_analyzer.py game.pgn -o analysis.tex

# Analyze with custom Stockfish path and deeper analysis
python chess_game_analyzer.py game.pgn -o analysis.tex \
    --stockfish /usr/local/bin/stockfish \
    --depth 22 \
    --time 2.0

# Analyze multiple games as a book
python chess_game_analyzer.py tournament.pgn -o book.tex --book \
    --book-title "My Tournament Games" \
    --book-author "Your Name"
```

4.4.2 Python API

```
from chess_game_analyzer import (
    EnhancedGameAnalyzer,
    EnhancedLaTeXReportGenerator,
    classify_game_character
)

# Analyze a game
with EnhancedGameAnalyzer("/usr/local/bin/stockfish",
                          depth=20, time_limit=1.0) as analyzer:
    result = analyzer.analyze_game("game.pgn")

# Access move-by-move data
for move in result.moves:
    pos = move.positional_eval
    print(f"Move {move.ply}: {move.move_san}")
    print(f"  Space: W={pos.space_white:.2f}, B={pos.space_black:.2f}")
    print(f"  Mobility: W={pos.mobility_white:.2f}, B={pos.mobility_black:.2f}")
    print(f"  Eval: {move.eval_after/100:+.2f} pawns")

# Generate LaTeX report
latex = EnhancedLaTeXReportGenerator.generate_report(
    result,
    include_plots=True,
    include_methodology=True
)
```

```
with open("analysis.tex", "w") as f:
    f.write(latex)
```

4.5 Positional Metrics Explained

4.5.1 Space

Space measures territorial control: central squares in your own half of the board that your pawns guard.

The computation counts squares on central files (c–f) in ranks 2–4 (for White) or 5–7 (for Black) that are:

1. Attacked by at least one friendly pawn, *and*
2. Not occupied by an enemy piece

The raw count is divided by 10 to normalize the metric. Values typically range from 0.1 to 0.7.

Example from Estrin vs Berliner (1965), ply 20 (10...e4):

- White Space: 0.30
- Black Space: 0.30

At this point in a sharp Two Knights Defense, space is roughly equal, but the position is about to explode tactically.

4.5.2 Mobility

Mobility adds 0.1 for every square attacked by one of your pieces (pawns included, the king excluded) that is not attacked by an enemy pawn. All piece types count equally, a square attacked by two of your pieces counts twice, and squares occupied by your own pieces count too. The metric captures piece activity and coordination.

Note on scale. In QARC reports made before October 2026, including the two cited in this manual, the Mobility values in the tables and plots are half of this value, because of a since-fixed bug. The raw data block at the end of each LaTeX file always stores the full value, and the numbers below use the full value.

Example from Estrin vs Berliner, ply 30 (15...Bxf3):

- White Mobility: 3.90
- Black Mobility: 5.00

After White's blunder 15.Be2??, Black's pieces have become significantly more active.

4.5.3 King Safety

King Safety combines two factors:

1. **Pawn shield**: +0.5 for each friendly pawn on the 2nd rank (7th for Black) on the king's file or an adjacent file

2. **Attacked squares**: −0.2 for each square within two squares of the king that an enemy piece attacks

Positive values indicate a safe king; negative values indicate danger.

Example 4.5.1. *From Estrin vs Berliner, ply 52 (26...hxg2+):*

- *White King Safety:* −1.40

- *Black King Safety:* −0.70

White's king is significantly more exposed after the pawn storm breaks through.

4.5.4 Threats

Threats evaluates the tactical tension in the position:

- **Hanging pieces**: Pieces attacked but not defended (weighted by piece value ×2)

- **Attacks by lower-value pieces**: e.g., a knight attacking a queen

- **King zone pressure**: Attacks on squares near the enemy king

- **Safe checks**: Available checking moves that don't lose material

- **Weak squares (holes)**: Squares that can never be defended by pawns

Raw Threats values typically range from 1 to 30. In the Fireteam Index calculations, Threats is scaled down (divided by 20) to be comparable with the other metrics.

Example 4.5.2. *From Estrin vs Berliner, ply 47 (24.Nc4): The Threats graph has spikes and valleys in this part of the game.*

- *White Threats: 21.80*

- *Black Threats: 8.50*

White has significant pressure, but Black's position is resilient.

4.6 The Fireteam Index (FTI)

4.6.1 Background: Chess Metrics and Win Prediction

Traditional chess analysis focuses on **centipawn evaluation**—a single number indicating who stands better. But this doesn't tell you *why* someone is winning or *how* the advantage manifests.

The **Fireteam Index** is inspired by Rithmomachia,[2] a medieval mathematical board game where one victory condition is establishing a "well-coordinated fireteam" of pieces deep in enemy territory forming arithmetic, geometric, or harmonic progressions.

Similarly, in chess, sustained positional dominance across multiple factors—space, activity, king safety, and threats—often precedes victory. The FTI combines these factors into a single metric that tracks this multi-dimensional advantage over time.

4.6.2 FTI Signatures

The module provides five FTI weightings, originally intended as "signatures" of different playing styles (a tactical player might win through direct threats, a positional player through space and piece activity). Chapter 3 found no evidence that they work that way.

FTI Variant	Space	Mobility	King Safety	Threats	Style
FTI_1 (Harmonious)	0.25	0.25	0.25	0.25	Balanced
FTI_2 (Tactical)	0.60	0.10	0.00	0.30	Space + threats
FTI_3 (Strategic)	0.70	0.10	0.10	0.10	Spatial domination
FTI_4 (Dynamic)	0.20	0.60	0.00	0.20	Piece activity
FTI_5 (Solid)	0.30	0.20	0.50	0.00	Defensive play

The FTI formula for a given player is:

$$\text{FTI} = w_S \cdot \Delta S + w_M \cdot \Delta M + w_K \cdot \Delta K + w_T \cdot \Delta T' \tag{4.1}$$

where each Δ is (Player's value − Opponent's value) and $T' = T/20$ (scaled for comparability).

4.6.3 Prediction Algorithms

Three algorithms use the FTI to predict game outcomes. Chapter 3 reports how well they work (not better than simple baselines, so far).

1. Per-Ply (Raw).

- Examines raw FTI at each ply after move 8 (ply 16)
- Predicts *win* if FTI $> \epsilon$ (default 0.2) for 10 consecutive plies
- Captures sudden, decisive advantages

[2]Rithmomachia, also known as the "philosopher's game," was a medieval mathematical board game popular in European monasteries and universities from the 11th to 16th centuries.

2. Windowed (Smoothed).

- Computes a 10-ply rolling average of FTI
- Applies the same streak detection to smoothed values
- Filters out tactical noise; captures sustained trends

3. Balance (Confidence-based).

- Counts plies where FTI $> \epsilon$ versus FTI $< -\epsilon$
- Measures volatility V: how many times the FTI switches between $> \epsilon$ and $< -\epsilon$
- Computes confidence $= |B_n| \times \max(0,\, 1 - V/V_{\text{thr}})$, where $V_{\text{thr}} = 0.2\times$ (number of plies considered)
- Predicts *win/loss* if confidence > 0.15; otherwise *draw*.
- Best at detecting drawn games and balanced struggles.

4.7 Concrete Example: Estrin vs Berliner (1965)

Let us walk through the analysis of a famous game from the 5th World Correspondence Chess Championship—**Yakov Estrin vs Hans Berliner**, featuring the sharp Two Knights Defense.

4.7.1 The Game (C57 – Two Knights Defense)

As we saw in the previous chapter, the first few moves of the game are as follows.

1.e4 e5 2.Nf3 Nc6 3.Bc4 Nf6 4.Ng5 d5 5.exd5 b5!?

Berliner plays the Ulvestad Variation (5...b5), a sharp gambit that had been analyzed deeply in correspondence play.

4.7.2 Analysis Output

Player Statistics.

Metric	Estrin (White)	Berliner (Black)
Accuracy	88.1%	92.1%
Avg centipawn loss	25.3	16.4
Best moves	14	16
Blunders	2	1

Game Character Classification.

- Evaluation range: $m_1 = -5.41$, $m_2 = +2.24$
- Spread: $d = 7.65$ pawns
- Classification: **Chaotic seesaw battle**

The game swung wildly—White had +2.24 at one point, but Black eventually achieved −5.41.

4.7.3 Positional Metrics Summary

Metric	White Average	Black Average
Space	0.30	0.22
Mobility	3.03	3.34
King Safety	−0.25	−0.22
Threats	6.35	5.91

Despite the sharp tactical nature, Berliner maintained slightly better piece activity throughout.

4.7.4 Move-by-Move Data Extract

The module stores raw positional data in a machine-readable format:

```
% ply, SAN, eval_cp, space_w, space_b, mob_w, mob_b, ks_w, ks_b,
%      threats_w, threats_b
% 29, Be2, -157, 0.300, 0.300, 5.000, 4.600, 0.000, 0.100, 3.800, 4.700
% 30, Bxf3, -212, 0.200, 0.300, 3.900, 5.000, -0.200, 0.500, 7.600, 6.000
% 31, Bxf3, -171, 0.300, 0.300, 4.000, 4.400, 0.200, 0.500, 7.400, 5.200
```

This data can be parsed for further computational analysis, machine learning, or statistical studies.

4.8 Command Line Interface

```
python chess_game_analyzer.py [PGN_FILE] [OPTIONS]
```

4.8.1 Core Options

Option	Description	Default
`-o, --output FILE`	Output LaTeX file path	none (prints a summary)
`--json-output FILE`	Output raw analysis as JSON	none
`-s, --stockfish PATH`	Path to Stockfish executable	`/usr/games/stockfish`
`-d, --depth N`	Analysis depth (higher = slower)	20
`-t, --time SECS`	Time limit per position	1.0
`-q, --quiet`	Suppress progress messages	false

4.8.2 Report Options

Option	Description
`--no-diagrams`	Don't include chess board diagrams
`--no-methodology`	Don't include methodology explanation section
`--no-plots`	Don't include matplotlib plots
`--ascii-plots`	Include ASCII plots (works without matplotlib)
`--plot-dir DIR`	Directory for plot output files

4.8.3 Fireteam Index Prediction (Single Game)

Option	Description
`--prediction W|B`	Include FTI prediction for White (W) or Black (B)
`--prediction-name NAME`	Player name for display (e.g., "Berliner")

4.8.4 Book Mode (Multiple Games)

Option	Description
`--book`	Analyze all games in PGN as a book
`--book-title TITLE`	Title for the book
`--book-author AUTHOR`	Author name
`--prediction-winner`	Track the winner of each decisive game (always on)
`--prediction-name NAME`	Track specific player by name
`--no-prediction`	Disable FTI prediction section entirely

4.8.5 Examples

```
# Deep analysis with plots
python chess_game_analyzer.py game.pgn -o report.tex \
    --stockfish /opt/stockfish/stockfish \
    --depth 24 --time 3.0
```

```
# Quick analysis without graphics
python chess_game_analyzer.py game.pgn -o report.tex \
    --depth 16 --time 0.5 --no-plots --ascii-plots

# Analyze Berliner's games as a collection
python chess_game_analyzer.py berliner_games.pgn -o berliner_book.tex \
    --book \
    --book-title "Berliner's World Championship Games" \
    --book-author "Analysis by Stockfish 17" \
    --prediction-name "Berliner"

# JSON output for programmatic access
python chess_game_analyzer.py game.pgn --json-output analysis.json
```

4.9 Python API

4.9.1 Main Classes

EnhancedGameAnalyzer. The core analysis engine. Use as a context manager to ensure proper cleanup.

```
with EnhancedGameAnalyzer(
    stockfish_path="/usr/local/bin/stockfish",
    depth=20,           # Search depth (plies)
    time_limit=1.0,     # Seconds per position
    extract_positional=True  # Compute Space/Mobility/etc.
) as analyzer:
    result = analyzer.analyze_game("game.pgn")
    # or analyze_all_games() for multi-game PGNs
```

EnhancedGameAnalysisResult. Contains all analysis data:

- `moves`: List of `EnhancedMoveAnalysis` objects
- `white_stats`, `black_stats`: Accuracy and move classification counts
- `brilliant_sacrifices`: Detected sacrifices
- `critical_positions`: Key moments for diagrams
- `game_character`: Classification (balanced/tactical/chaotic, etc.)
- `positional_summary`: Aggregated Space/Mobility/etc. statistics

PositionalEvaluation. Attached to each move, containing:

- `space_white`, `space_black`
- `mobility_white`, `mobility_black`
- `king_safety_white`, `king_safety_black`
- `threats_white`, `threats_black`

4.9.2 Utility Functions

```
from chess_game_analyzer import (
    parse_raw_positional_data,
    compute_fireteam_index,
    predict_outcome_per_ply,
    predict_outcome_windowed,
    predict_outcome_balance,
    classify_game_character,
    FTI1_WEIGHTS
)

# Parse raw data from a CGA-generated LaTeX file (pass the file name)
data = parse_raw_positional_data("berliner_book.tex", game_num=7)

# FTI with the FTI1 weights, as in the reports (Threats divided by 20).
# The prediction functions below compute the FTI themselves from ‘data‘.
fti_data = compute_fireteam_index(data, weights=FTI1_WEIGHTS)

# Run predictions with custom weights
from chess_game_analyzer import FTI2_WEIGHTS  # Tactical signature

result = predict_outcome_per_ply(
    data,
    player_color="B",
    player_name="Berliner",
    weights=FTI2_WEIGHTS,
    cutoff_ply=16,
    streak_length=10,
    epsilon=0.2
)
print(f"Prediction: {result.prediction}")
print(f"Max streak: {result.max_streak} plies")
```

4.10 Parameter Reference

4.10.1 Analysis Parameters

Parameter	Type	Default	Description
`stockfish_path`	str	`/usr/games/stockfish`	Path to Stockfish binary
`depth`	int	20	Search depth in plies
`time_limit`	float	1.0	Maximum seconds per position
`min_diagram_spacing`	int	6	Min ply distance between diagrams
`top_n_swings`	int	2	"Biggest swing" positions to include

4.10.2 FTI Prediction Parameters

Parameter	Type	Default	Description
`cutoff_ply`	int	16	Opening plies to skip
`streak_length`	int	10	Consecutive plies for WIN
`epsilon`	float	0.2	Min FTI for "meaningful advantage"
`window_size`	int	10	Rolling average window
`weights`	tuple	$(0.25, 0.25, 0.25, 0.25)$	(w_S, w_M, w_K, w_T)
`v_ratio`	float	0.20	Volatility threshold ratio
`confidence_threshold`	float	0.15	Min confidence for WIN/LOSS

4.10.3 Available Weight Presets

```
from chess_game_analyzer import (
    FTI1_WEIGHTS,  # (0.25, 0.25, 0.25, 0.25) - Harmonious
    FTI2_WEIGHTS,  # (0.60, 0.10, 0.00, 0.30) - Tactical
    FTI3_WEIGHTS,  # (0.70, 0.10, 0.10, 0.10) - Strategic
    FTI4_WEIGHTS,  # (0.20, 0.60, 0.00, 0.20) - Dynamic
    FTI5_WEIGHTS,  # (0.30, 0.20, 0.50, 0.00) - Solid
)
```

4.10.4 Move Classification Thresholds

Classification	Centipawn Loss
Book	under 30 cp, and within the first 12 plies
Best	0–5 cp
Excellent	5–15 cp
Good	15–30 cp
Inaccuracy	30–60 cp
Mistake	60–120 cp
Blunder	120+ cp

4.11 Output Formats

4.11.1 LaTeX Report

The primary output is a LaTeX document that can be compiled with `pdflatex` or `xelatex`. It includes:

1. **Game Information**: Event, players, date, opening
2. **Player Statistics**: Accuracy, move counts by classification
3. **Positional Analysis**: Space/Mobility/KingSafety/Threats summaries with plots
4. **Game Character Classification**: Balanced/tactical/chaotic assessment
5. **Annotated Game**: Move-by-move with NAG symbols and comments
6. **Critical Positions**: Diagrams at key moments with best alternatives
7. **Methodology Section**: (optional) Detailed explanation of all metrics
8. **Fireteam Index Prediction**: (optional) Win prediction analysis with plots; in book mode, also held-out tests of the FTI (see chapter 3)

Required LaTeX packages: `xskak`, `chessboard`, `amsmath`, `amssymb`, `geometry`, `longtable`, `booktabs`, `xcolor`, `pgfplots`, `graphicx`, `hyperref`.

4.11.2 JSON Output

Raw analysis data in JSON format for programmatic access:

```
{
  "white": "Estrin",
  "black": "Berliner",
  "result": "0-1",
  "moves": [
    {
      "ply": 1,
      "move_san": "e4",
      "eval_after": 15,
      "classification": "best",
      "positional_eval": {
        "space_white_mg": 0.40,
        "space_black_mg": 0.40,
        "mobility_white_mg": 3.90,
        ...
      }
    },
```

```
    ...
  ],
  "white_stats": {"accuracy": 88.1, ...},
  "black_stats": {"accuracy": 92.1, ...}
}
```

4.11.3 Raw Positional Data

Appended after `\end{document}` in LaTeX files, machine-readable CSV format:

```
% GAME_DATA_START
% 1, e4, 15, 0.400, 0.400, 3.900, 3.300, 1.000, 1.500, 0.800, 0.800
% 2, e5, 24, 0.400, 0.400, 3.900, 3.900, 1.000, 1.000, 0.800, 0.800
...
% GAME_DATA_END
```

Fields: ply, SAN, eval_cp, space_w, space_b, mob_w, mob_b, ks_w, ks_b, threats_w, threats_b.

4.12 License

This software is released under the **MIT License** (see the file `LICENSE.txt` in the repository).

Copyright © 2026, David Joyner

4.13 Acknowledgments

- **Stockfish** (`https://stockfishchess.org/`) — The chess engine powering the evaluations
- **python-chess** (`https://python-chess.readthedocs.io/`) — The excellent chess library
- The concept of the Fireteam Index is inspired by **Rithmomachia**, the medieval "philosopher's game"

Chapter 5

References

Bibliography

[Du17] A. Dunne, *Hans Berliner, 1929-2017*, Chess Life, May 2017, pp. 43–45.

[Be65] *Quantitative-Analytic Reports on Chess: Berliner's games, 5th World Ch. ICCF 1965-1968* (Generated by Stockfish 16, depth 20, on October 18, 2026), the QARC pdf of Berliner's games in the 5th World Correspondence Chess Championship, ICCF, 1965. Available at:
`https://github.com/wdjoyner/qarc`

[Be99] H. Berliner, **The System: A World Champion's Approach to Chess**, Gambit Publ., 1999.

[Be05] Interview with Gardner Hendrie on 2005-03-07, *Oral history of Hans Berliner*, CHM Reference number X3131.2005, Computer History Museum, 2005.

[CC18] *Quantitative-Analytic Reports on Chess: Carlsen vs Caruana, 2018 World Championship, London* (Generated by Stockfish 16, depth 20, on October 18, 2026), the QARC pdf of all 12 games in the 2018 World Chess Championship, FIDE. Available at:
`https://github.com/wdjoyner/qarc`

[CGA26] D. Joyner, the Chess Game Analyzer. Available at:
`https://github.com/wdjoyner/chess-animator`

[FEN] S. J. Edwards, *Portable Game Notation Specification and Implementation Guide*, 1994. Section 16.1 defines FEN, which extends a notation devised by D. Forsyth. Available at:
`https://github.com/fsmosca/PGN-Standard/blob/master/PGN-Standard.txt`

[PV] *Chess piece relative value*, wikipedia. Available at:
`https://en.wikipedia.org/wiki/Chess_piece_relative_value`

[Ri25] David Joyner, *Rithmomachia: is recreational arithmetic*, monograph, 2025. Available at `amazon.com` or in the docs directory at:
`https://github.com/wdjoyner/rithmomachia`

[To12] J. Togyer, *The Iconoclast*, in **The Link: The Magazine of the Carnegie Mellon University School of Computer Science**, May 7, 2012.